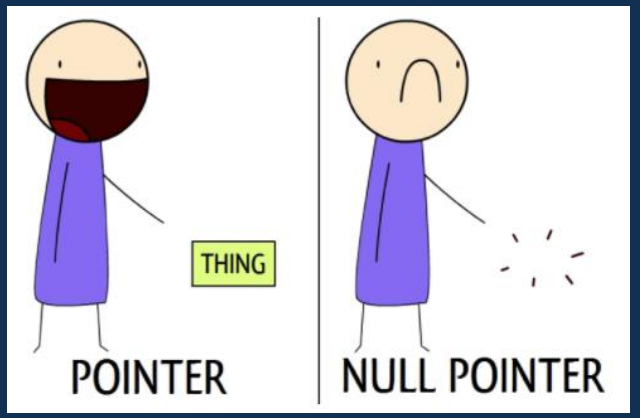
01. Pointers Revisited

Kyu-haeng Lee

SW Convergence Coding II

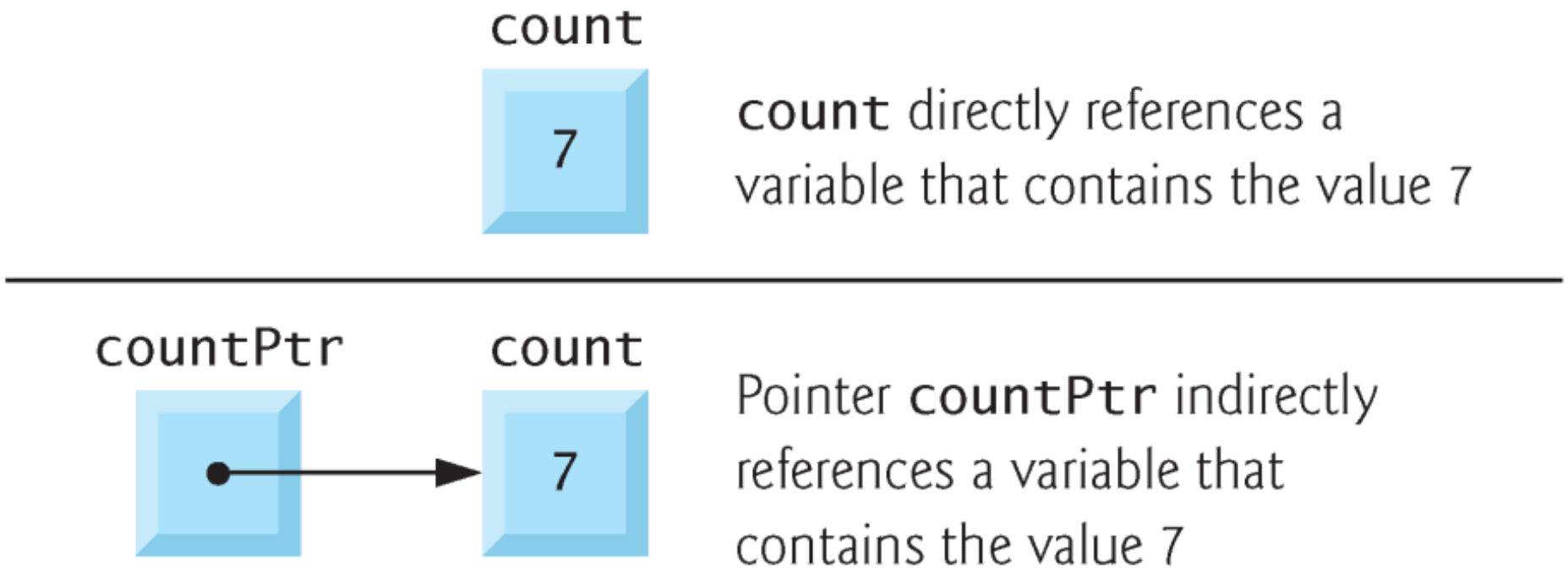
Fall 2025



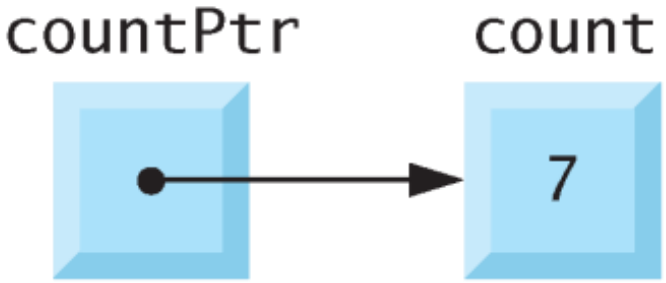
Pointer Basics

What's a Pointer?

- Pointer: Variable whose value is a memory address.
  - In general, a variable directly contains a specific value.
  - A pointer, on the other hand, contains an address of a variable that contains a specific value.



Pointer Definition



- int *countPtr, count;
  - specifies that variable countPtr is of type int *
  - i.e., a pointer to an integer and is read, “countPtr is a pointer to int” or “countPtr points to an object of type int.”

  - The * only applies to countPtr in the definition.
    - The variable count is defined to be an int, not a pointer to an int.

Pointer Initialization

- A pointer may be initialized to NULL, 0 or an address.
  - NULL: a symbolic constant
  - Initializing a pointer to 0 is equivalent to initializing a pointer to NULL, but NULL is preferred.

- Null Pointer
  - A pointer whose value is set as NULL.
  - Means the pointer isn't assigned any valid memory address yet.
  - Very common to check whether a pointer is NULL, before using it.
    - if (ptr == NULL)

Pointer Operators

- & (address operator)
  - unary operator that returns the address of its operand.

- Example
  - int y = 5, *yPtr;
  - yPtr = &y;
  - assigns the address of the variable y to pointer variable yPtr.



Pointer Operators

- * operator
  - Indirection operator or dereferencing operator
  - Returns the value of the object to which its operand (i.e., a pointer) points.

- Example
  - printf("%d", *yPtr);
  - prints the value of variable y, namely 5.
  - Using * in this manner is called dereferencing a pointer.



Pointer Operators

- * is used in three different ways in C/C++.
  - 1. Multiplication: c = a * b;
  - 2. Pointer declaration: int *ptr;
  - 3. Dereferencing a pointer: c = a + *ptr;

Example

```c
/* Fig. 7.4: fig07_04.c
   Using the & and * operators */
#include <stdio.h>

int main( void )
{
   int a; /* a is an integer */
   int *aPtr; /* aPtr is a pointer to an integer */

   a = 7;
   aPtr = &a; /* aPtr set to address of a */

   printf( "The address of a is %p"
           "\nThe value of aPtr is %p", &a, aPtr );

   printf( "\n\nThe value of a is %d"
           "\nThe value of *aPtr is %d", a, *aPtr );

   printf( "\n\nShowing that * and & are complements of "
           "each other\n&*aPtr = %p"
           "\n*&aPtr = %p\n", &*aPtr, *&aPtr );
   return 0; /* indicates successful termination */
} /* end main */
```

```
The address of a is 0012FF7C
The value of aPtr is 0012FF7C

The value of a is 7
The value of *aPtr is 7

Showing that * and & are complements of each other.
&*aPtr = 0012FF7C
*&aPtr = 0012FF7C
```

Call-by-Value and Call-by-Reference

- Call-by-Value
  - A copy of the argument's value is made and passed to the called function.
  - Changes to the copy do not affect an original variable's value in the caller.

- Call-by-Reference
  - The caller allows the called function to modify the original variable's value.

Call-by-Value and Call-by-Reference

```
void call_by_val(int a) {
    a = a * a;
}

void call_by_addr(int* a) {
    *a = (*a) * (*a);
}

// only available in C++
void call_by_ref(int& a) {
    a = a * a;
}
```

```
int main() {
    int a = 2;
    call_by_val(a);
    printf("call_by_val: %d₩n", a);

    call_by_addr(&a);
    printf("call_by_addr: %d₩n", a);

    call_by_ref(a);
    printf("call_by_ref: %d₩n", a);

    return 0;
}
```

Call-by-Value and Call-by-Reference

- Important Note
  - All function calls in C programs are by value.
  - In C++, call-by-reference is supported.

  - In C, the effect of call-by-reference is obtained by passing pointers as arguments. ➔ call-by-address

sizeof Operator

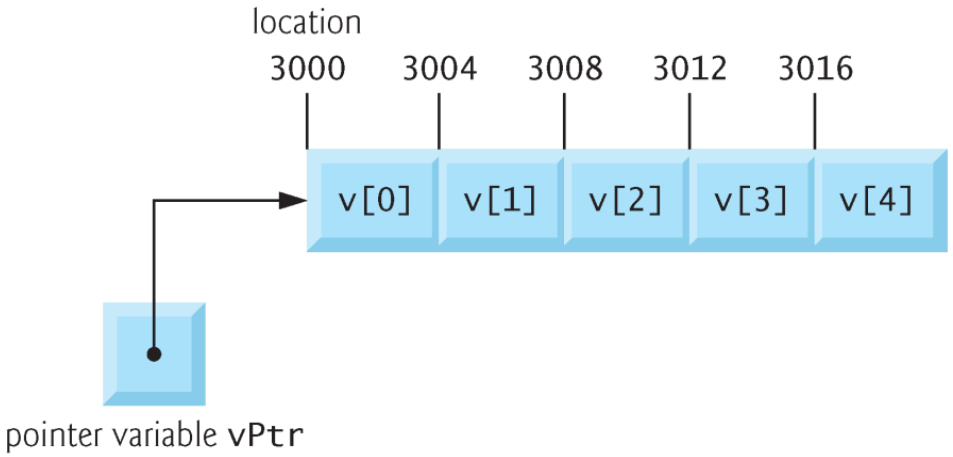
- sizeof: determines the size in bytes of data (type) during program compilation.
  - Type size_t == unsigned int

```
1  /* Fig. 7.16: fig07_16.c
2     Applying sizeof to an array name returns
3     the number of bytes in the array. */
4  #include <stdio.h>
5
6  size_t getSize( float *ptr ); /* prototype */
7
8  int main( void )
9  {
10    float array[ 20 ]; /* create array */
11
12    printf( "The number of bytes in the array is %d"
13            "\nThe number of bytes returned by getSize is %d\n",
14            sizeof( array ), getSize( array ) );
15    return 0; /* indicates successful termination */
16 } /* end main */
17
18 /* return size of ptr */
19 size_t getSize( float *ptr )
20 {
21    return sizeof( ptr );
22 } /* end function getSize */
```

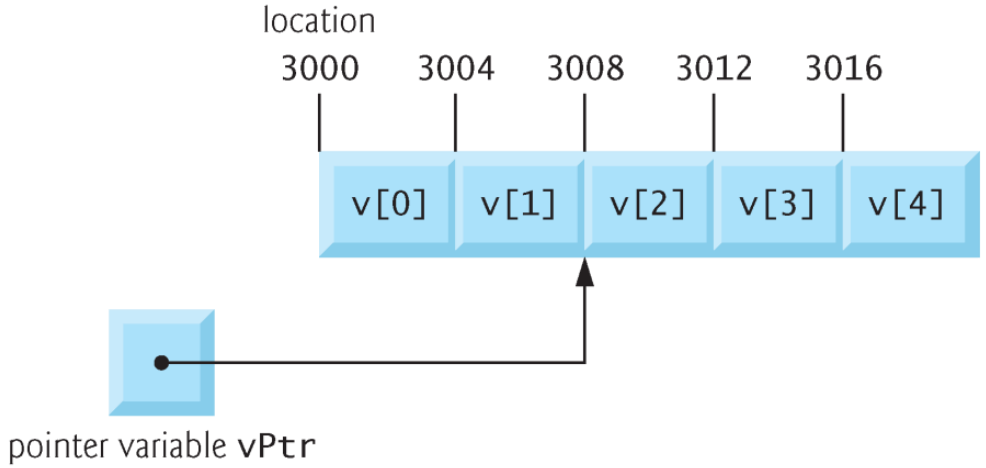
```
The number of bytes in the array is 80
The number of bytes returned by getSize is 4
```

Pointer Arithmetic

- Some arithmetic operations can be performed on pointers.
  - +, -, ++, --



vPtr = v;



vPtr += 2;

Pointer Arithmetic

- Either of the statements
  - ++vPtr;
    vPtr++;

  - increments the pointer to point to the next location in the array.

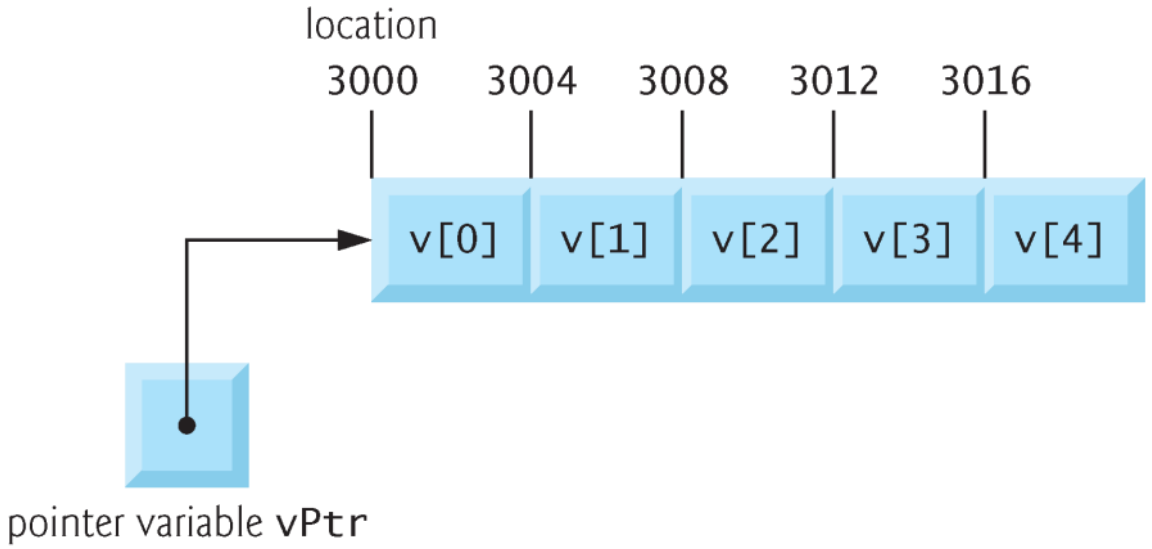
- The same applies to -- operator.

Pointer Arithmetic

- Pointer variables may be subtracted from one another.
- Example
  - vPtr: contains the address 3000
  - v2Ptr: contains the address 3008

  - x = v2Ptr - vPtr;

  - would assign to x the number of array elements from vPtr to v2Ptr, i.e., 2 (not 8).



void Pointer

- void *
  - A generic pointer that can represent any pointer type.
  - A cast operation is required for dereferencing.

```
void* gp;
int a = 1;
char b[] = "string";

gp = &a;
printf("%d", *(int *)gp);

gp = b;
printf("%s", (char*)gp);
```

Updated 25/09/08

Exercise 1

- Fix the following code.

```
int main() {
    int a = 10;
    char b = 'b';
    char* c = "abc";
    void* p = NULL;
    p = &a;
    printf("int a: %d₩n", p);   //display the value of a.
    p = &b;
    printf("char b: %d₩n", p);  //value of b
    p = &c;
    printf("string c: %d₩n", p);    //value of c
    return 0;
}
```

void Pointer

- Example

```
typedef struct {
    double x;
    double y;
} Point;

int main() {
    double a[5] = { 0, 1, 2, 3, 4 };
    int b[5] = { 5, 6, 7, 8, 9 };
    Point ps[2] = { {0.5, 0.5},
                    {1.5, 2.5} };

    gp(a, 5, sizeof(double));
    gp(b, 5, sizeof(int));
    gp(ps, 2, sizeof(Point));
}
```

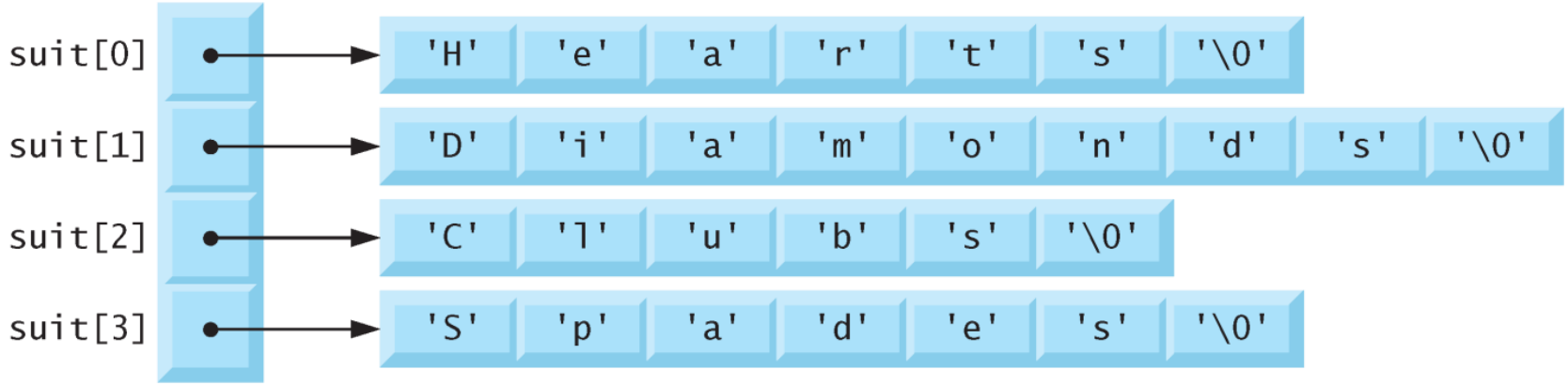
```
void gp(void* b, size_t n, size_t size) {
    char* p;
    for (p = b; p < (char*)(b + (n * size)); p += size) {
        switch (size) {
            case sizeof(double) :
                printf("%.2f ", *(double*)p);
                break;
            case sizeof(int) :
                printf("%d ", *(int*)p);
                break;
            case sizeof(Point) :
                printf("x = %.2f ", ((Point*)p)->x);
                printf("y = %.2f ", ((Point*)p)->y);
                break;
        }
    }
    printf("₩n");
}
```

Pointer and Array

- Array of pointers
  - This is an array; each element contains a pointer variable.
  - Syntax: type *array-name[size];
    - int *arr[4]; //arr is an array of four pointers to int

- Pointer to an array
  - This is a pointer; it moves by the array size at a time.
  - Syntax: type (*pointer-name)[size];
    - int (*ptr)[4]; //ptr is a pointer to an array of size four.

Array of Pointers

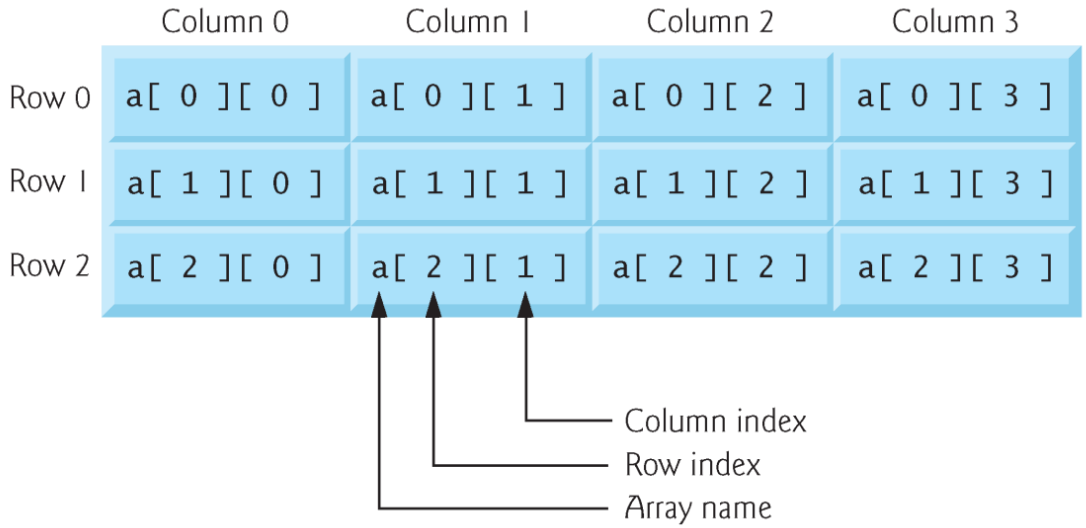


- Declared as char *suit[4];
- Four pointers are in the array; each points to a different address.
  - The pointer in the first element points to the beginning of string "Hearts\0".

Pointer to an Array

- int (*ptr)[4];
  - can point to any arrays of size four.
  - ptr = &a[0]; //ptr points to the first row of a.
    - Each row can be viewed as an array

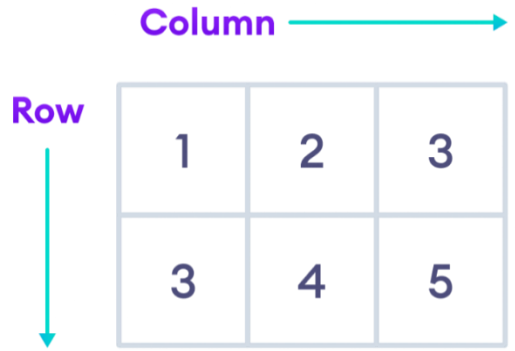
- Dereferencing
  - (*ptr)[0] // a[0][0]
  - (*(ptr+1))[2] // a[1][2]
    - Jump 16B (the size of one row)



In a double-subscripted array, the first row is stored in memory followed by the second row; each row is basically a single-subscripted array.

Exercise 2

- Using a pointer to an array, write code that prints two rows each.




```
int main() {
    int arr[2][3] = { 1,2,3,3,4,5 };

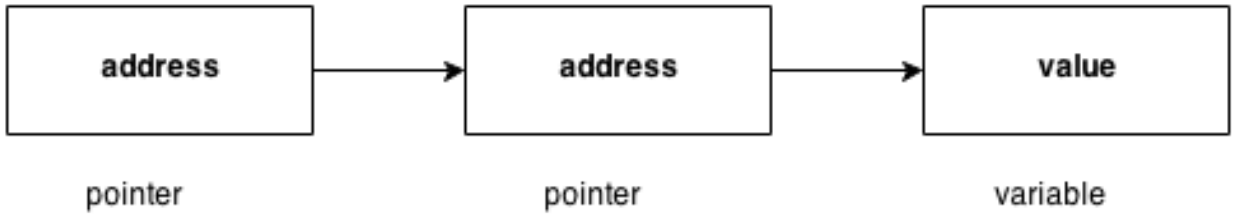
    //your code goes here.

    return 0;
}
```

Pointer to Pointer

Pointer to Pointer

- C allows a pointer to point to another pointer.
  - Just add more *s before the name in the declaration.



  - int **ptr;
    - ptr is called a double pointer. It can point to a pointer to int.

  - More complicated pointers…
    - int *******ptr; // ptr is a pointer that points to a pointer that points to a pointer that …

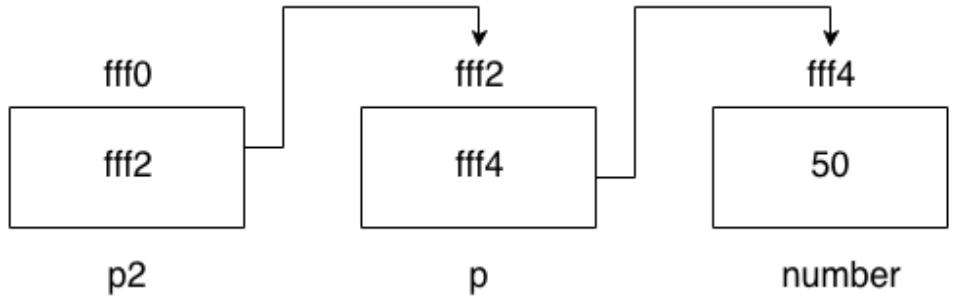
Pointer to Pointer

- Example of Double Pointers

```
int main() {
    int number = 50;
    int* p;          //pointer to int
    int** p2;        //pointer to pointer
    p = &number;     //stores the address of number variable
    p2 = &p;

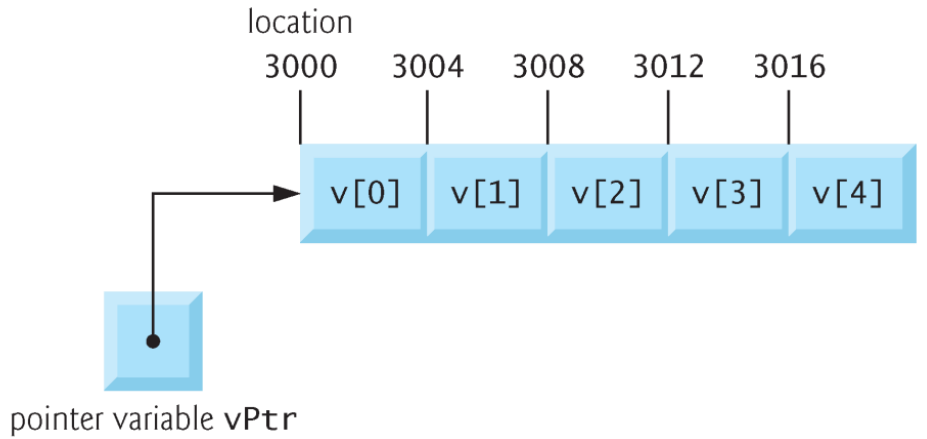
    printf("Address of number variable is %p ₩n", &number);
    printf("Address of p variable is %p ₩n", p);
    printf("Value of *p variable is %d ₩n", *p);
    printf("Address of p2 variable is %p ₩n", p2);
    printf("Value of **p2 variable is %d ₩n", **p2);

    return 0;
}
```



Pointer and Array

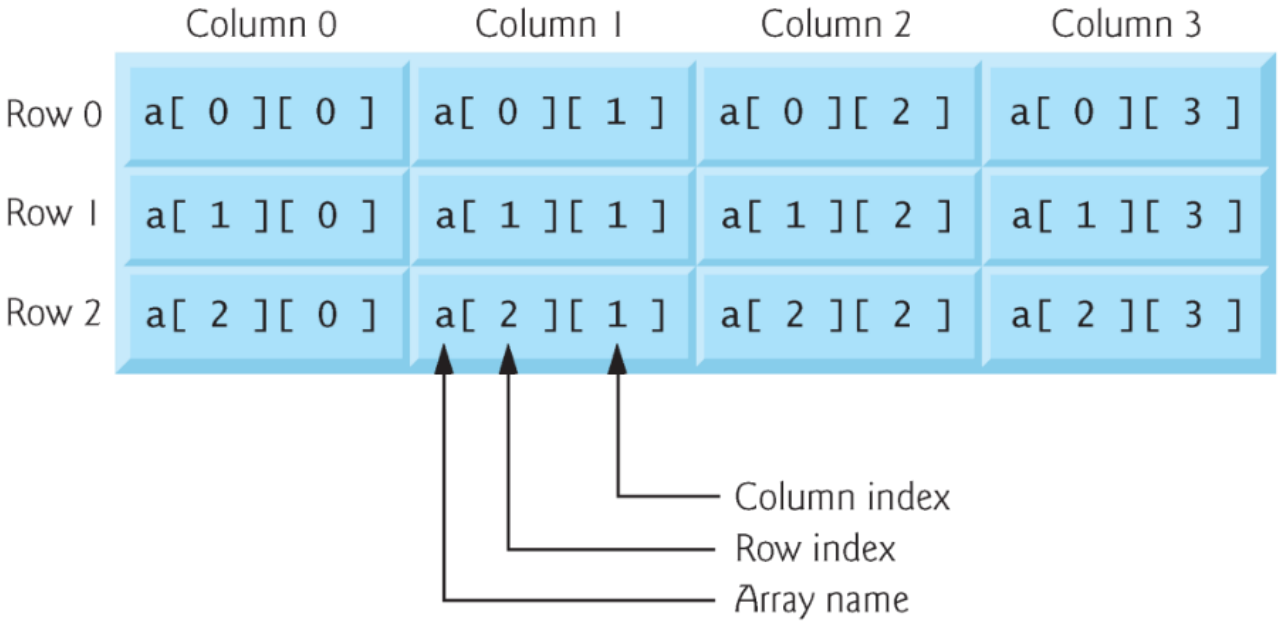
- We know
  - Single pointers work well with 1D arrays
  - Ex)
    - vPtr = &v[2];
    - vPtr++;
  - For an 1D array, to access array elements:
    - a[i] = *(a+i)



- What about multi-dimensional arrays?

2D Arrays

- int a[m][n];
  - Total size: m x n x sizeof(int)
  - Think of
    - a[0] as row 0
    - a[1] as row 1
    - …
- For a 2D array,
  - a[i][j] = *(*(a+i)+j)

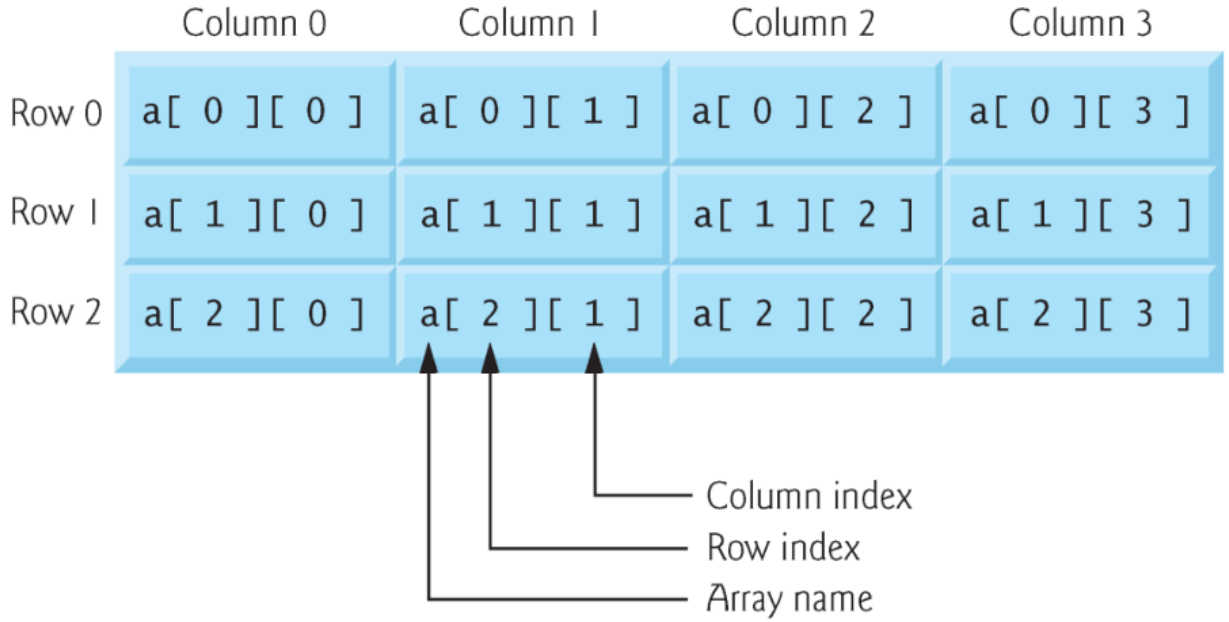


  - Recall: For an 1D array, to access array elements:
    - a[i] = *(a+i)

2D Arrays

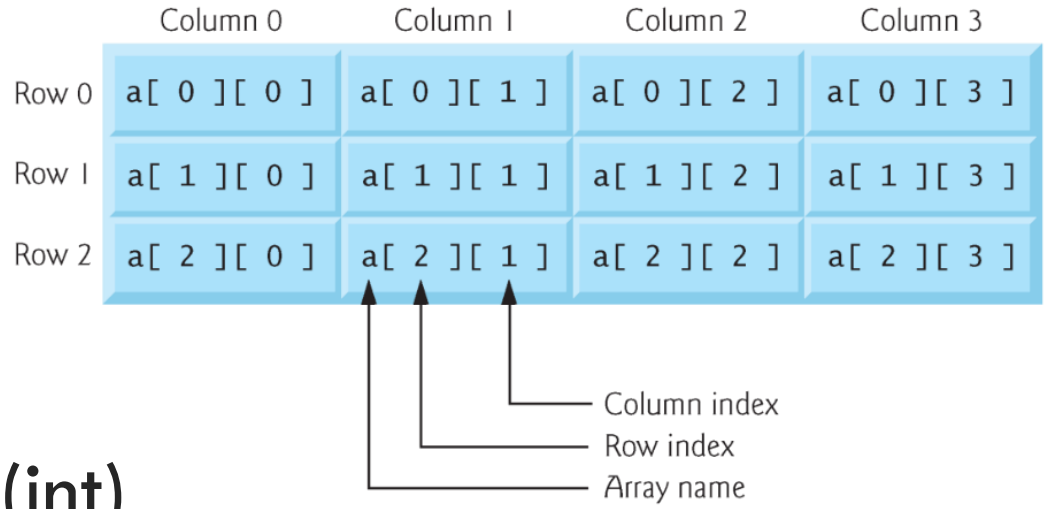
- For a 2D array,
  - a[i][j] = *(*(a+i)+j)
  - a[i][j] = *(a[i]+j) = *(*(a+i)+j)

  - Example: a[0][2] = *(a[0] + 2)
  - a[0][0] = **a



2D Arrays

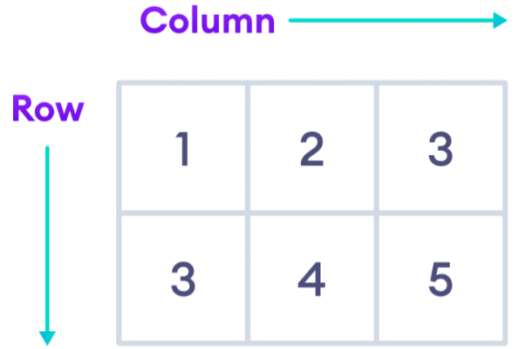
- For a 2D array int a[m][n];
- address(a[i][j])
  = address(a[0][0]) + (i x n + j) x size(int)



- &a[i][0] = &(*(a[i]+0)) = &*a[i] = a[i]
  - All these things indicate the same address.

Exercise 3

- Write code that displays the address of each element.
  - Define a pointer p to point to the first element
  - Write a printf with a pointer expression



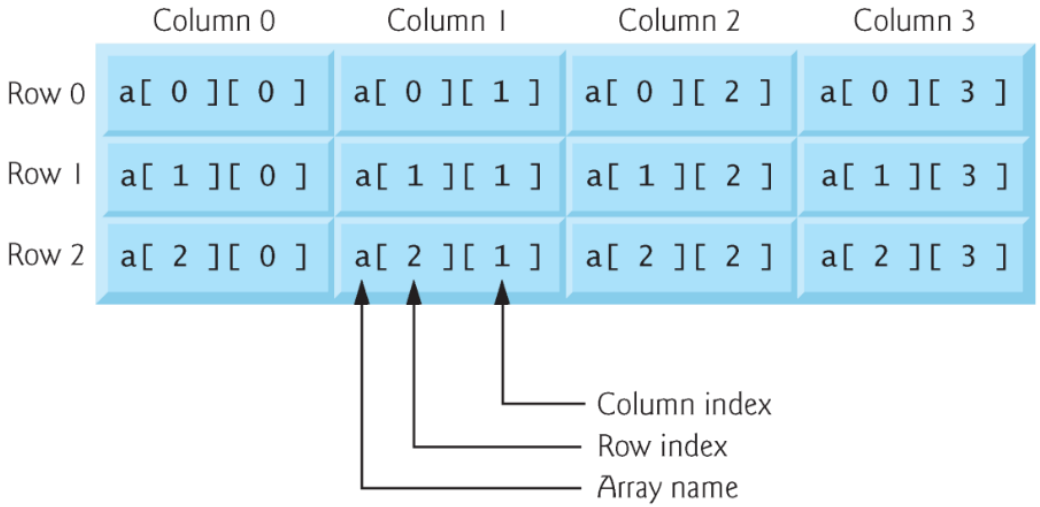

```
int main() {
    int arr[2][3] = { 1,2,3,3,4,5 };

    //your code goes here.

    return 0;
}
```

2D Arrays

- For a 2D array int a[m][n];
- a: pointer to row 0
  - Type: int (*)[n]



- Dereferencing a: *a
  - Row 0 or a[0]
  - a[0] has type int *

- Dereferencing a[0]: *a[0]
  - The first element of row 0 or a[0][0] or **a
  - **a is an int.

Updated 25/09/08

2D Arrays

- Different types
  - &a: type int (*)[m][n], pointer to the entire 2D array
  - &a[0]: type int (*)[n], pointer to first row
    - considered same as a
  - &a[0][0]: type int *, pointer to first element of row 0
  - a: type int(*)[n], (decayed) pointer to first row
  - *a: type int*, pointer to first element of first row
    - type: arr[n] → treated as first row itself.
  - a[0]: same as *a
  - *a[0]: type int, value of the first element, i.e., a[0][0]

- 2D array
  - is NOT equivalent to a double pointer!
  - is “equivalent” to a “pointer to row”.

```
a == &a[0]
*a == a[0]
**a == a[0][0]
```

Passing a 2D Array to a Function

```
int main()
{
    int A[3][3], i, j;
    for (i = 0; i < 3; i++)
        for (j = 0; j < 3; j++)
            A[i][j] = i * 10 + j;

    for (i = 0; i < 3; i++) {
        printf("₩n");
        for (j = 0; j < 3; j++)
            printf("%4.2d", A[i][j]);
    }
    printf("₩n");
    f1(A);
    f2(A);
    f3(A);
    f4(A);
    f5(A);
    return 0;
}
```

Passing a 2D Array to a Function (1)

- Declare as matrix, explicitly specify second dimension
- Don't have to specify the first dimension!

```
void f1(int A[][3]) {
    int i, j;
    for (i = 0; i < 3; i++) {
        printf("₩n");
        for (j = 0; j < 3; j++)
            printf("%4.2d", A[i][j]);
    }
    printf("₩n");
}
```

Passing a 2D Array to a Function (2)

- A pointer to array, second dimension is explicitly specified

```
void f2(int(*A)[3]) {
    int i, j;
    for (i = 0; i < 3; i++) {
        printf("₩n");
        for (j = 0; j < 3; j++)
            printf("%4.2d", A[i][j]);
    }
    printf("₩n");
}
```

Passing a 2D Array to a Function (3)

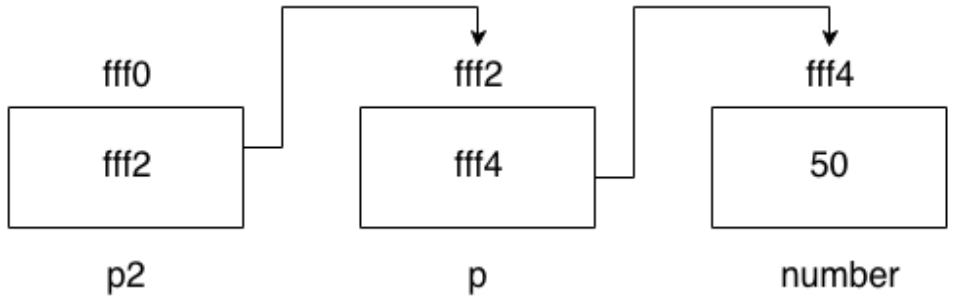
- Using a single pointer, the array is "flattened"

```
void f3(int* A) {
    int i, j;
    for (i = 0; i < 3; i++) {
        printf("₩n");
        for (j = 0; j < 3; j++)
            printf("%4.2d", *(A + 3 * i + j));
    }
    printf("₩n");
}
```

Passing a 2D Array to a Function (4)

- A double pointer, using an auxiliary array of pointers
- Add the dimensions to the formal argument list if you allocate "index" at run-time.

```
void f4(int** A) {
    int i, j, * index[3];
    for (i = 0; i < 3; i++)
        index[i] = (int*)A + 3 * i;
    for (i = 0; i < 3; i++) {
        printf("₩n");
        for (j = 0; j < 3; j++)
            printf("%4.2d", index[i][j]);
    }
    printf("₩n");
}
```



Passing a 2D Array to a Function (5)

- A pointer to array, using an auxiliary array of pointers

```
void f5(int(* A)[3]) {
    int i, j, * index[3];
    for (i = 0; i < 3; i++)
        index[i] = (int*)A + 3 * i;
    for (i = 0; i < 3; i++) {
        printf("₩n");
        for (j = 0; j < 3; j++)
            printf("%4.2d", index[i][j]);
    }
    printf("₩n");
}
```

Function Pointers

Function Pointers

- A pointer can point to a function (its starting addr)

- void (*pf)(int);
  - pf is a pointer to a function that takes one int parameter and returns void:
  - If f is such a function, we can make pf point to f in the following way: pf = f;
  - We can now call f by writing either
    - (*pf)(i); or pf(i);

Function Pointers

- An array whose elements are function pointers:

```
void (*file_cmd[])(void) = {        new_cmd,
                                    open_cmd,
                                    close_cmd,
                                    close_all_cmd,
                                    save_cmd,
                                    save_as_cmd,
                                    save_all_cmd,
                                    print_cmd,
                                    exit_cmd
};
```

  - A call of the function stored in position n of the file_cmd array:
    - (*file_cmd[n])(); /* or file_cmd[n](); */
  - We could get a similar effect with a switch statement, but using an array of function pointers provides more flexibility.

Example

- No need for &printf or (*fp)
- Note: … in the printf prototype is a variable argument.
  - It takes variable number of arguments.
    - ex) printf(“%d %d %d %d”, a, b, c, d);

```
int main() {
    int i = 1;
    int (*fp) (const char*, ...) = printf;
    fp("i == %d₩n", i);
    (*fp)("i == %d₩n", i);

    return 0;
}
```

Example

```
Enter a number between 0 and 2, 3 to end: 0
You entered 0 so function1 was called

Enter a number between 0 and 2, 3 to end: 1
You entered 1 so function2 was called

Enter a number between 0 and 2, 3 to end: 2
You entered 2 so function3 was called

Enter a number between 0 and 2, 3 to end: 3
Program execution completed.
```

```c
/* Fig. 7.28: fig07_28.c
   Demonstrating an array of pointers to functions */
#include <stdio.h>

/* prototypes */
void function1( int a );
void function2( int b );
void function3( int c );

int main( void )
{
   /* initialize array of 3 pointers to functions that each take an
      int argument and return void */
   void (*f[ 3 ])( int ) = { function1, function2, function3 };

   int choice; /* variable to hold user's choice */

   printf( "Enter a number between 0 and 2, 3 to end: " );
   scanf( "%d", &choice );

   /* process user's choice */
   while ( choice >= 0 && choice < 3 ) {

      /* invoke function at location choice in array f and pass
         choice as an argument */
      (*f[ choice ])( choice );

      printf( "Enter a number between 0 and 2, 3 to end: " );
      scanf( "%d", &choice );
   } /* end while */

   printf( "Program execution completed.\n" );
   return 0; /* indicates successful termination */
} /* end main */

void function1( int a )
{
   printf( "You entered %d so function1 was called\n\n", a );
} /* end function1 */

void function2( int b )
{
   printf( "You entered %d so function2 was called\n\n", b );
} /* end function2 */

void function3( int c )
{
   printf( "You entered %d so function3 was called\n\n", c );
} /* end function3 */
```

Exercise

- Fill in the blanks to complete the following code.

```
#pragma warning (disable:4996)
#include <stdio.h>

int add(int a, int b) { return a + b; }
int sub(int a, int b) { return a - b; }

int main() {
    //TO DO 1. Your code goes here.

    int a = 1, b = 2;
    printf("%d + %d = %d₩n", a, b, fp(1, 2));

    //TO DO 2. Your code goes here.

    printf("%d - %d = %d₩n", a, b, fp(1, 2));
    return 0;
}
```

Deciphering Complex Declarations

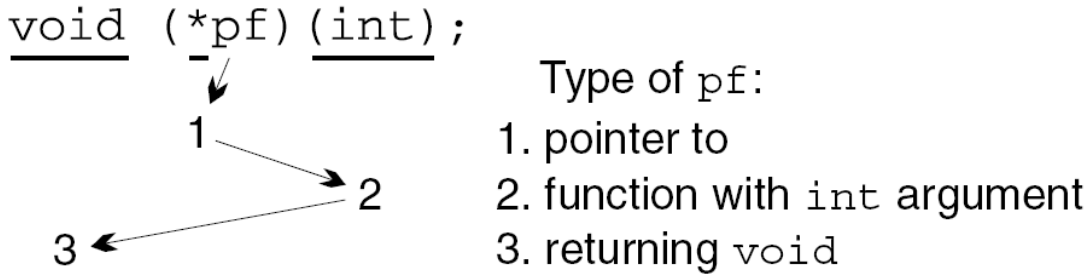
Deciphering Complex Declarations

- What does the following declaration mean?
  - int *(*x[10])(void);
  - x is a pointer, or an array, or a function?

Right-Left Rule

- Steps
  - Step 0. begins with the var name.
  - Step 1. read right
  - Step 2. read left if there's nothing to read or) is encountered.
  - Step 3. repeat from step 1, if there's nothing to read or (is encountered.

- Readings for symbols
  - *: pointer to
  - []: array of
  - (): function returning



Right-Left Rule

- **int *a[]**
  - **a**: a is
  - **[]**: an array of
  - *: pointers to
  - **int**: int

- **int (*a)[]**
  - **a**: a is
  - *: a pointer to
  - **[]**: an array of
  - **int**: ints

Right-Left Rule

- **int *(*func())();**
  - **func**: func is
  - **()**: a function returning
  - *: a pointer to
  - **()**: a function returning
  - *: a pointer to
  - **int**: int

References

- Book
  - Chapter 7. Pointers, C: How to Program, Paul Deitel et al.
  - Chapter 5. Pointers and Arrays, The C Programming Language Prentice Hall Software, Kernighan, Brian W. et al.

- Right-Left Rule
  - C Right-Left Rule (Rick Ord's CSE 30 - UC San Diego) (ucsd.edu)
  - https://cseweb.ucsd.edu//~ricko/rt_lt.rule.html